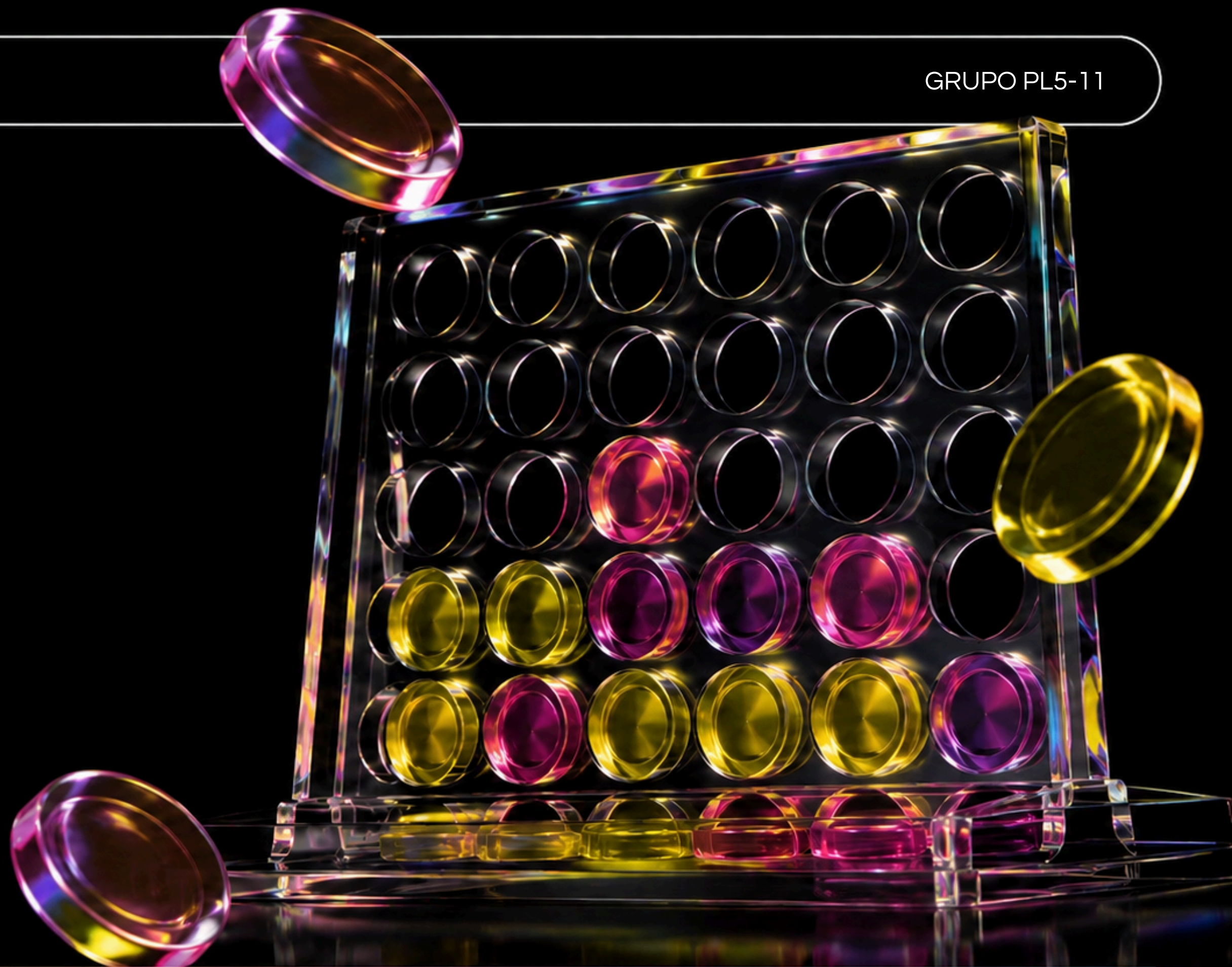


POP OUT



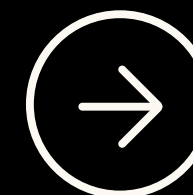
USER x USER
MCTS
ID3



INTRODUÇÃO

Este projeto tem como objetivo implementar o jogo PopOut, uma variação do Connect 4, e aplicar técnicas de Inteligência Artificial para tomada de decisão durante as partidas.

A proposta combina duas abordagens: o uso do Monte Carlo Tree Search para pesquisar jogadas por meio de simulações e o uso do algoritmo ID3 para aprender decisões a partir de um dataset gerado automaticamente.



O QUE É POP OUT?

O PopOut é uma variação do Connect 4 em que os jogadores podem não apenas inserir peças no tabuleiro (drop), mas também remover peças pela parte inferior das colunas (pop).

Essa mecânica torna o jogo mais dinâmico, porque aumenta significativamente as possibilidades de jogada.

CONNECT 4

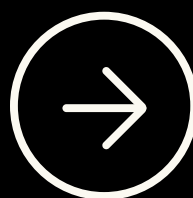
- Só drop
- Peças entram por cima
- Jogo mais previsível

POPOUT

- Drop e pop
- Peças também podem sair por baixo
- Mais estados e reversões



ESTRUTURA DO PROJETO



POPOUT.PY

Ficheiro que contém o jogo PopOut pronto para ser executado e jogado.

MCTS.PY

Ficheiro responsável pela IA baseada em Monte Carlo Tree Search.

ID3.PY

Ficheiro responsável pela árvore de decisão, treinada com datasets gerados.

DATA/

Pasta onde ficam os datasets utilizados no projeto (gerados pelo MCTS e o dataset Iris).

IMPLEMENTAÇÃO E INTERFACE

```
PopOut
```

```
1- USER vs USER
```

```
2- USER vs IA
```

```
> 
```

```
-----  
O-----  
X-OX---  
OXXOXX-  
OXOXXO
```

```
It is now O's turn. Enter move (drop/pop + column):
```

```
O> 
```

```
Escolha a IA:
```

```
1- MCTS 100 iterações
```

```
2- MCTS 3000 iterações
```

```
3- MCTS 5000 iterações
```

```
4- ID3 treinado com dataset_popout_3000.csv
```

```
> 
```

```
OXXOXXO  
XXXXXX  
OXXOXO  
XOOOXO  
XXXOXXO
```

```
It is now X's turn. Enter move (drop/pop + column):
```

```
O tabuleiro está cheio! Podes fazer um 'pop' ou declarar empate.
```

```
Queres terminar com empate? (sim/nao): 
```


POPOUT.PY

PopOut.py é o ficheiro central do projeto. Ele contém a versão executável do jogo e concentra a lógica necessária para manter o estado do tabuleiro, validar movimentos, identificar estados terminais e integrar as IAs ao modo de jogo.

ESTADO E MOVIMENTOS

- board
- current_player
- state_history
- last_move_type
- last_player

Responsáveis por guardar o estado atual, controlar turnos e permitir regras como empate por repetição.

VALIDAÇÃO DE JOGADAS

- is_column_full()
- is_valid_drop()
- is_valid_pop()
- get_legal_moves()

get_legal_moves() é essencial para as IAs, pois devolve todas as ações possíveis no formato:

[('drop', coluna), ('pop', coluna)]

ESTADOS TERMINAIS

- get_winners()
- get_game_result()
- board_full()

Aqui ficam as verificações mais importantes:

Vitória de X ou O, vitória dupla após pop, empate por repetição, tabuleiro cheio.

CHECK_WINNER_FOR() VS GET_WINNERS()

A maior melhoria de desempenho veio da otimização da verificação de vitória, uma função chamada milhares de vezes durante as simulações do MCTS.

ANTES: CHECK_WINNER_FOR()

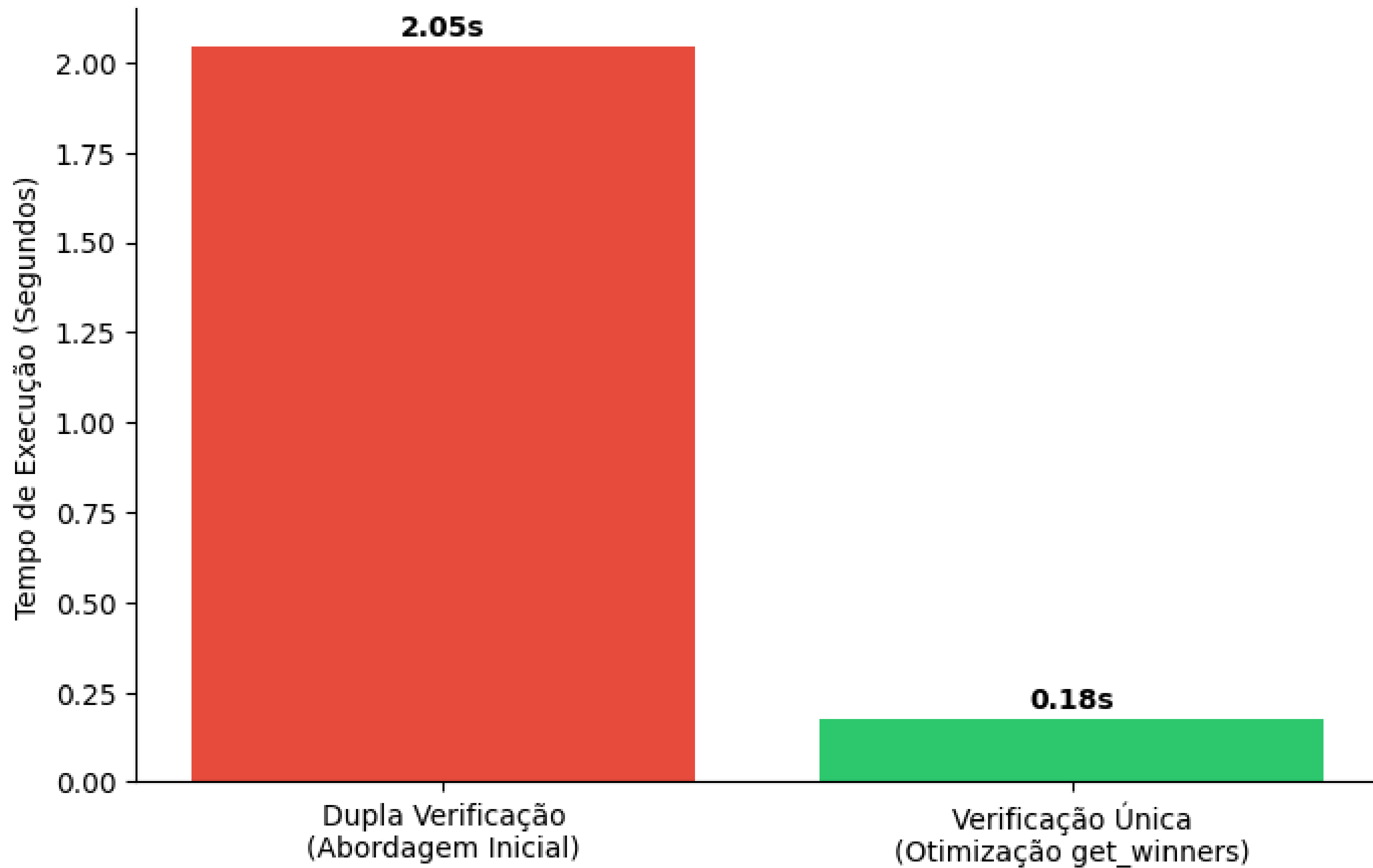
- Verificava X e O separadamente
- Duas varreduras no tabuleiro
- Uso de all() e generator expressions
- Alto custo dentro do MCTS

DEPOIS: GET_WINNERS()

- Uma única varredura no tabuleiro
- Identifica X, O ou ambos ao mesmo tempo
- Comparações diretas
- Mesma lógica, menor custo

Não alterámos a regra do jogo, apenas tornámos a verificação de estados terminais mais eficiente.

Impacto da Otimização do Algoritmo de Verificação de Vitórias



MCTS.PY

A IA MCTS foi implementada em duas classes principais: Node e MCTS.

Cada nó representa um estado possível do jogo e guarda a jogada que levou até ele, os filhos explorados, o número de visitas e as vitórias acumuladas.

No método `search()`, o algoritmo clona o estado atual, percorre a árvore com Upper Confidence Bounds for Trees (UCT), expande novas jogadas, simula partidas até o fim e retropropaga o resultado.

O estado do jogo é clonado em cada simulação, através da função `clone()`, para evitar alterar o tabuleiro real da partida.

Também foi implementado o método `update_root()`, que permite reaproveitar parte da árvore entre turnos, evitando reconstruir toda a pesquisa após cada jogada.

$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\ln N(PARENT(n))}{N(n)}}$$

GERAÇÃO DO DATASET COM MCTS

A função `generate_dataset(self, num_games, iterations)` cria exemplos de treino para o ID3 a partir de partidas simuladas entre duas IAs MCTS.

O QUE A FUNÇÃO FAZ:

- Cria partidas simuladas automaticamente
- Usa duas IAs MCTS, uma para X e outra para O
- Em cada turno, grava:
 - estado do tabuleiro
 - jogada escolhida pelo MCTS
- Atualiza a árvore com `update_root()`
- Guarda os dados em `dataset_popout_<iterations>.csv`

```
A iniciar a geração de 3 jogos (iterações: 3000)
A simular jogo 1 de 3...
A simular jogo 2 de 3...
A simular jogo 3 de 3...
```

```
Geração concluída! Ficheiro 'dataset_popout_3000.csv' criado/atualizado com sucesso.
Tempo Total de execução 57.95
```


IMPLEMENTAÇÃO DO ID3

Implementámos o algoritmo ID3 manualmente e depois aplicámos ao PopOut usando o dataset gerado pelo MCTS.

1. VALIDAÇÃO INICIAL

Antes de aplicar ao PopOut, validámos o ID3 com o iris.csv.

Objetivo: Confirmar se a árvore conseguia aprender padrões simples e fazer previsões coerentes.

sepalength: cortes em [5.4, 5.8]
sepalwidth: cortes em [2.9, 3.4]
petallength: cortes em [2.45, 4.8]
petalwidth: cortes em [0.8, 1.7]

== Classificação de Novos Exemplos ==

PL=1.4 PW=0.2 → Iris-setosa
PL=4.5 PW=1.5 → Iris-versicolor
PL=5.5 PW=2.1 → Iris-virginica

Resultado obtido com 90% de eficácia.

2. APLICAÇÃO AO POPOUT

Dataset MCTS 3000



Tabuleiro 6x7 transformado em 42 atributos



Jogada MCTS como classe a prever



Árvore ID3 treinada

3. USO DURANTE O JOGO

Estado atual do tabuleiro



ID3 percorre a árvore

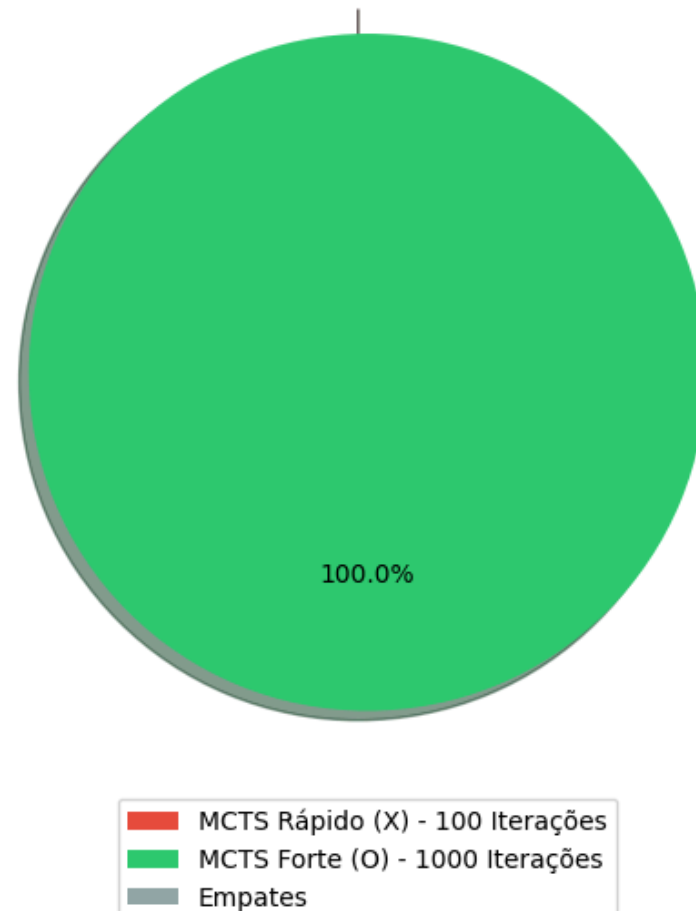


Retorna uma jogada: drop coluna ou pop coluna

O ID3 não simula partidas durante o jogo: ele usa a árvore treinada para decidir rapidamente com base em exemplos aprendidos do MCTS.

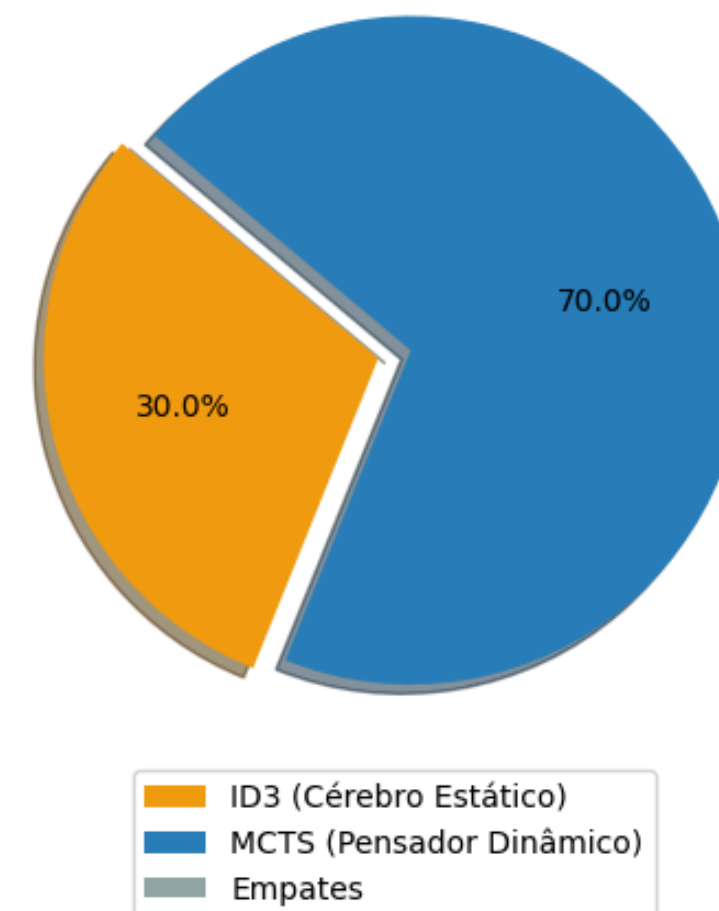
TESTES E COMPARAÇÃO ENTRE IAS

Taxa de Vitória: Quantidade vs Qualidade de Simulações



MCTS vs MCTS: A profundidade vence a rapidez (1000 iter vs 100 iter)

Duelo Final: ID3 vs MCTS



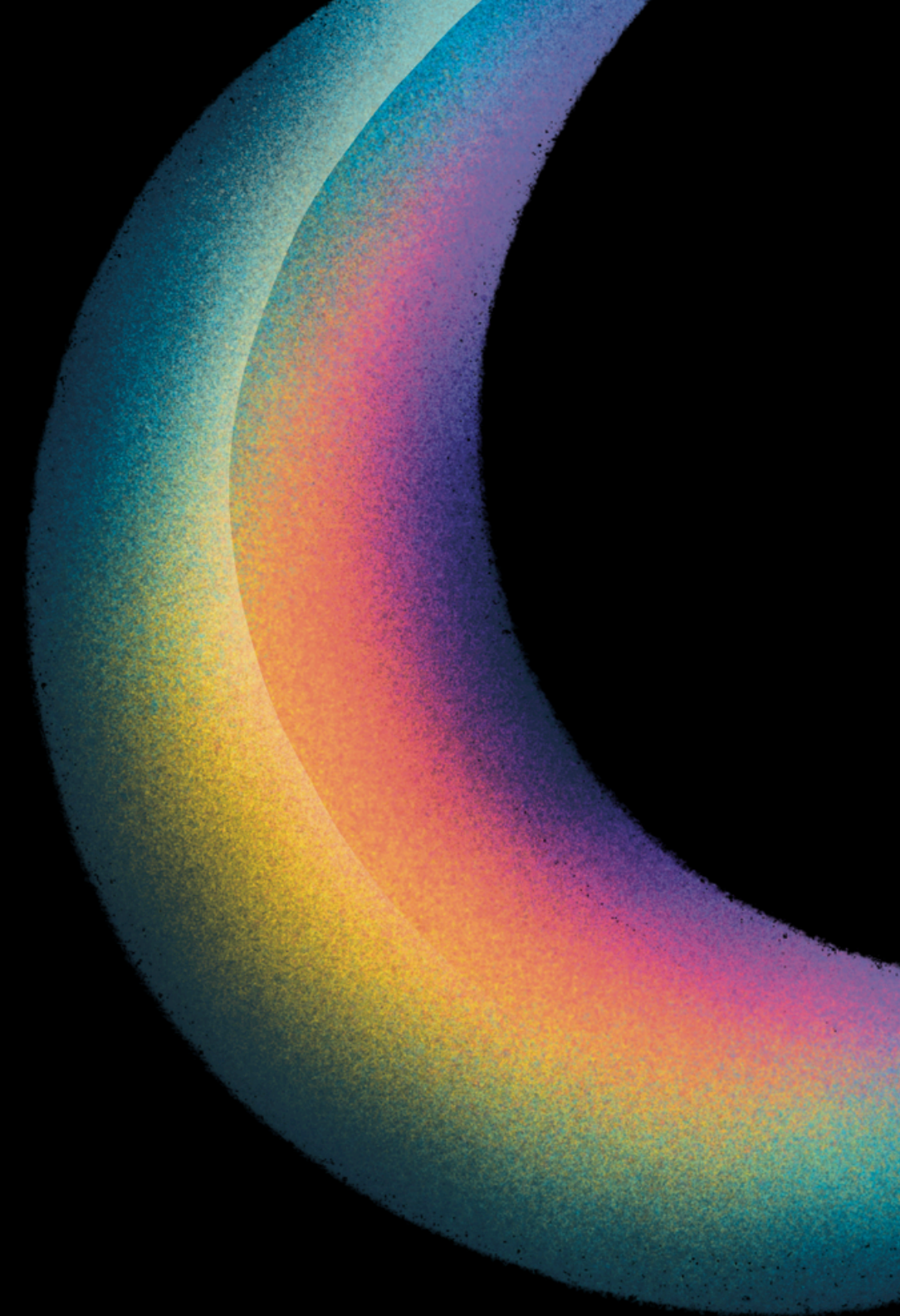
O ID3 treinou o dataset em ~0,18 segundos e foi capaz de vencer partidas contra o MCTS. Isso mostra que conseguimos transferir parte da estratégia aprendida pelo MCTS para uma árvore estática, permitindo decisões em milissegundos.

PROBLEMAS ENCONTRADOS E DECISÕES TÉCNICAS

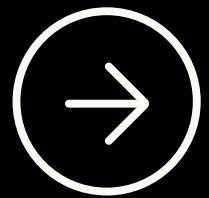
A árvore memorizou perfeitamente os exemplos, mas falha em estados nunca antes vistos. Se o ID3 tentar uma jogada ilegal, intercetamos o erro e forçamos a uma jogada aleatória legal.

```
#Proteção contra ilegalidades devido a cenários nunca antes vistos
if not success:
    import random
    print(f"IA {game.current_player} tentou uma jogada inválida: {move}")
    move = random.choice(game.get_legal_moves())
    game.apply_move(move[0], move[1])
```

Amnésia do MCTS: O algoritmo original esquecia a árvore a cada turno. Implementamos a função `update_root()` com o objetivo de aumentar a eficiência do MCTS, reaproveitando a árvore de pesquisa das jogadas anteriores como suporte para as novas.



CONCLUSÃO



MCTS: Elevada adaptabilidade a jogar em tempo real, porém com custo computacional alto e mais tempo de resposta

ID3: Execução instantânea, mas sempre dependente da qualidade e quantidade do dataset. Um ID3 alimentado com dados excelentes supera um MCTS com poucas iterações.

Concluindo se tivermos grande poder e tempo de processamento, o MCTS é a melhor escolha devido à sua adaptabilidade. Mas se precisarmos de uma IA para correr em sistemas limitados e retornar respostas quase idênticas, porém instantâneas, o ID3 é a melhor opção.

OBRIQADO.